

GELİŞMİŞ RAG SİSTEMLERİ VE DEĞERLENDİRME METRİKLERİ

LlamaIndex ve TruLens ile Gelişmiş RAG Teknikleri Çalışma Raporu

Yazar: Gemini Notebook RAG Çalışma Grubu | Kaynak: DeepLearning.AI | Tarih: 17 Ağustos 2026

1. Giriş ve Temel RAG (Retrieval-Augmented Generation) Mimarisi

RAG (Geri Çağırma ile Güçlendirilmiş Üretim) sistemleri, büyük dil modellerinin (LLM) kendi eğitim verilerinde bulunmayan özel veya güncel dokümanlar üzerinde doğru yanıtlar vermesini sağlamak amacıyla kullanılan en popüler mimaridir. Temel bir RAG akışı üç ana aşamadan oluşur: **Veri Yükleme ve İndeksleme (Ingestion)**, **Geri Çağırma (Retrieval)** ve **Sentez (Synthesis)**.

- Ingestion (Veri Yükleme):** Dokümanlar sisteme yüklenir (load), küçük metin parçalarına (text chunks) bölünür ve her bir parça için bir gömme vektörü (embedding) oluşturularak vektör veri tabanına (index) kaydedilir.
- Retrieval (Geri Çağırma):** Kullanıcının gönderdiği sorgu (query) vektörleştirilir ve veri tabanındaki en benzer en yakın top-K parça (chunk) bulunarak geri çağırılır.
- Synthesis (Sentez):** Geri çağırılan ilgili metin parçaları ile kullanıcının orijinal sorusu birleştirilerek LLM'in prompt penceresine gönderilir ve modelin doğru, kaynağa dayalı bir yanıt üretmesi sağlanır.

GÜNLÜK HAYAT BENZETMESİ: GELENEKSEL KÜTÜPHANE ARAŞTIRMASI

Elinizde büyük bir ansiklopedi seti (veri kaynağı) olduğunu hayal edin. Bir konuda araştırma yapmanız gerektiğinde, kütüphaneci ilgili konunun geçtiği 3 farklı sayfayı fotokopi çekip önünüze getiriyor (Retrieval). Siz de bu sayfaları okuyup kendi cümlelerinizle sorunun yanıtını yazıyorsunuz (Synthesis). Ancak kütüphaneci sayfayı tam ortasından kestiği için konunun başını veya sonunu kaçırebiliyorsunuz. Temel RAG sistemlerindeki en büyük problem de tam olarak budur: Sabit boyutlu metin parçaları bağlam bütünlüğünü bozabilir.

Temel RAG Pipeline Kod Örneği (LlamaIndex):

```
# LlamaIndex ile Temel RAG Kurulumu ve Sorgulama
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader, ServiceContext
from llama_index.llms.openai import OpenAI

# 1. Dokümanların Yüklenmesi
documents = SimpleDirectoryReader(input_files=["career_in_ai.pdf"]).load_data()

# 2. LLM ve Gömme (Embedding) Modelinin Tanımlanması
# OpenAI GPT-3.5-Turbo ve HuggingFace BGEsmall modeli kullanılır
service_context = ServiceContext.from_defaults(
    llm=OpenAI(model="gpt-3.5-turbo"),
    embed_model="local:BAAI/bge-small-en-v1.5"
)

# 3. Chunking, Embedding ve İndeksleme İşlemlerinin Yapılması
index = VectorStoreIndex.from_documents(documents, service_context=service_context)

# 4. Sorgu Motorunun Oluşturulması ve Çalıştırılması
query_engine = index.as_query_engine()
response = query_engine.query("What are steps to take when finding projects to build your experience?")
print(response)
```

2. RAG Değerlendirme Üçlüsü (RAG Triad of Metrics)

Geliştirdiğimiz RAG sisteminin ne kadar iyi çalıştığını ölçmek için pahalı insan değerlendirmelerine veya sabit veri kümelerine bağımlı kalmak yerine, **LLM tabanlı değerlendirme metrikleri** kullanılır. TrueLens platformu ile geliştirilen **RAG Değerlendirme Üçlüsü (RAG Triad)**, üç kritik metriğin ikili karşılaştırmasına dayanır:

- **Groundedness (Dayanaklılık):** Üretilen yanıtın, veri tabanından getirilen bağlam (retrieved context) ile ne kadar desteklendiğini ölçer. Yanıtın uydurma (halüsinasyon) olup olmadığını denetler.
- **Context Relevance (Bağlam Uygunluğu):** Getirilen metin parçalarının (context) kullanıcının sorduğu soru ile ne kadar alakalı olduğunu ölçer. Geri çağırma mekanizmasının kalitesini gösterir.
- **Answer Relevance (Yanıt Uygunluğu):** Üretilen yanıtın, kullanıcının orijinal sorusuna doğrudan ve net bir şekilde cevap verip vermediğini ölçer.

GÜNLÜK HAYAT BENZETMESİ: ÖĞRETMEN, ÖĞRENCİ VE SINAV KİTAPÇIĞI

Bir öğrenci (LLM) açık kitap sınavına girmektedir. Sınav kitapçığından (Veri Tabanı) sorulan soruya göre öğrenci bazı sayfaları (Context) açar. 1) Öğrencinin açtığı sayfalar soruyla ne kadar alakalı? (Context Relevance). 2) Öğrencinin kağıda yazdığı cevap, açtığı bu sayfalara gerçekten dayanıyor mu yoksa kafasından mı uydurmuş? (Groundedness). 3) En önemlisi, öğrencinin yazdığı cevap hocanın sorduğu soruyu gerçekten yanıtlıyor mu? (Answer Relevance).

TruLens ile RAG Değerlendirme Üçlüsü Kurulumu:

```
# TruLens Değerlendirme Modüllerinin Başlatılması ve Kayıt İşlemi
from trulens_eval import Tru, Feedback, TruLlama
from trulens_eval.feedback.provider.openai import OpenAI as TruOpenAI

tru = Tru()
tru.reset_database()

# TruLens Recorder ile temel RAG query_engine'in sarmalanması
tru_recorder = TruLlama(
    query_engine,
    app_id="RAG_Baseline_v1",
    feedbacks=get_feedbacks() # Hazır triad değerlendirme fonksiyonlarını döndüren yardımcı fonksiyon
)

# Sorunun değerlendirme kapsamında çalıştırılması
with tru_recorder as recording:
    response = query_engine.query("What is the right AI job for me?")
```

3. Gelişmiş Teknik 1: Cümle Penceresi Geri Çağırma (Sentence-Window Retrieval)

Temel RAG sistemleri, indekslenen büyük metin parçalarını aynen geri çağırır. Ancak büyük metin parçaları bazen alakasız bilgiler içererek LLM'in kafasını karıştırabilir ve bağlam alakasını (context relevance) düşürebilir. **Sentence-Window Retrieval** bu sorunu çözmek için geliştirilmiştir:

- Metinleri indekslerken sadece ****tekil cümleler**** (bireysel küçük parçalar) şeklinde böler ve gömer. Arama işlemi bu nokta atışı küçük cümleler üzerinde yapılır.
- Geri çağırma (retrieval) gerçekleştikten sonra, bulunan hedef cümlelerin ****önündeki ve arkasındaki cümle pencereleri**** (örneğin pencere boyutu = 3) otomatik olarak eklenir.

- Böylece arama işlemi hassas ve küçük vektörlerle yapılırken, LLM'e giden bağlam tam ve geniş tutulmuş olur. Bu durum hem ****Dayanaklılığı (Groundedness)**** hem de ****Bağlam Uygunluğunu**** artırır.

GÜNLÜK HAYAT BENZETMESİ: CİMBİZLA ÇEKİP BÜYÜTEÇLE OKUMAK

Büyük bir romandan bir kelime veya cümle arıyorsunuz. Cimbizla tam aradığınız cümlelerin bulunduğu satırı çekip çıkarıyorsunuz (Hassas Geri Çağırma). Ancak cümleyi tek başına okuduğunuzda yazarın ne demek istediğini tam anlayamıyorsunuz. Bu yüzden büyütecinizi cümlelerin üzerine tutup, onun öncesindeki ve sonrasındaki 3'er satırı da içine alacak şekilde okuyorsunuz (Metadata Replacement). Böylece bağlam bütünlüğü harika şekilde korunuyor.

Sentence-Window Retrieval Kurulumu:

```
# LlamaIndex ile Sentence-Window Yapısı Kurulumu
from llama_index.core.node_parser import SentenceWindowNodeParser
from llama_index.core.postprocessor import MetadataReplacementPostprocessor

# 1. Cümle bazlı indeksleme için özel düğüm ayrıştırıcı
node_parser = SentenceWindowNodeParser.from_defaults(
    window_size=3, # Hedef cümlelerin önüne ve arkasına 3 cümle ekler
    window_metadata_key="window",
    original_text_metadata_key="original_text"
)

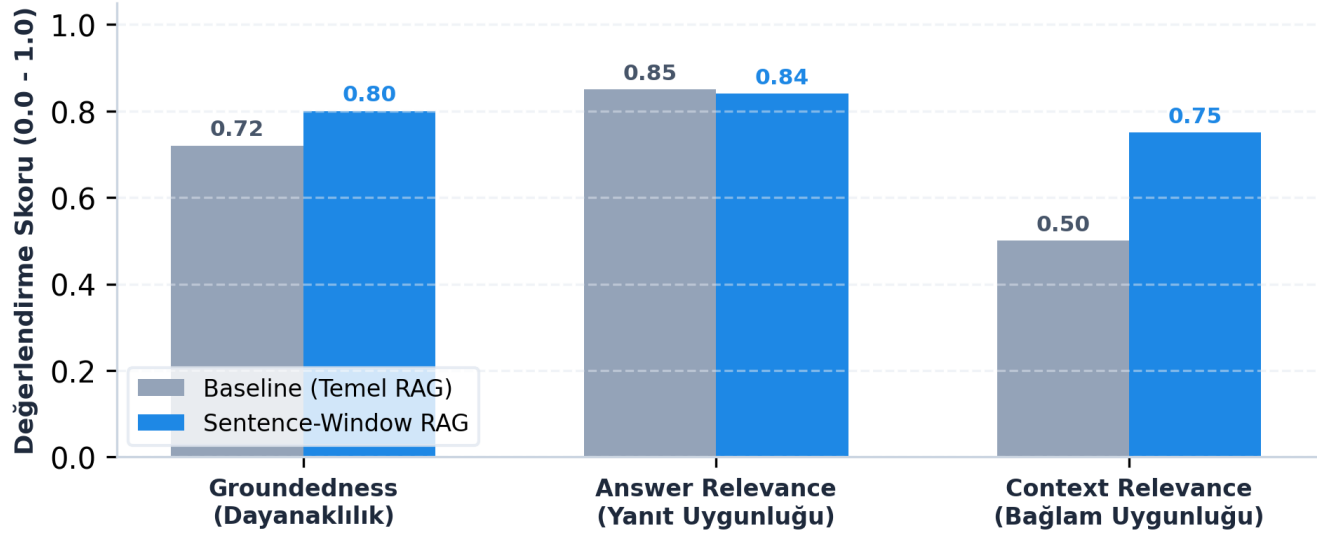
# 2. Geri çağırma aşamasında hedef cümleyi geniş pencere ile değiştiren post-processor
postprocessor = MetadataReplacementPostprocessor(target_metadata_key="window")

# Sorgu motoruna postprocessor'ın eklenmesi
query_engine = sentence_window_index.as_query_engine(
    node_postprocessors=[postprocessor]
)
```

Performans Değerlendirmesi:

TruLens liderlik tablosu (leaderboard) verilerine göre, **Sentence-Window Retrieval** mimarisi, Temel RAG baseline mimarisine kıyasla **Groundedness (Dayanaklılık)** skorunda %8'lik (8 yüzde puanı) doğrudan artış sağlamaktadır. Bununla birlikte, bağlam kalitesinin yükselmesi sayesinde **Context Relevance (Bağlam Uygunluğu)** bariz bir şekilde artarken, gereksiz büyük metin parçaları LLM'e gönderilmediği için **toplam token maliyeti (total cost)** daha düşük gerçekleşmektedir. Aşağıdaki grafik, TruLens ile yapılan bu değerlendirmenin skor karşılaştırmasını göstermektedir:

Temel RAG vs. Sentence-Window RAG Performans Karşılaştırması



4. Gelişmiş Teknik 2: Otomatik Birleştirme Geri Çağırma (Auto-Merging Retrieval)

Dokümanlar hiyerarşik bir yapıda düzenlendiğinde arama performansı optimize edilebilir. **Auto-Merging Retriever**, metinleri birbirine bağlı üst-alt düğüm (parent-child) hiyerarşisi şeklinde düzenler:

- Metinler önce büyük ana parçalara (örneğin 512 tokenlik Parent düğümler) bölünür. Ardından her bir ana parça daha küçük alt parçalara (örneğin 128 tokenlik 4 adet Child düğüm) ayrıştırılır.
- Arama işlemi en küçük seviyedeki Child düğümleri üzerinde gerçekleştirilir.
- Eğer geri çağrılan Child düğümlerinin **çoğunluğu** (örn. 4 alt düğümün 3'ü) aynı Parent düğümüne aitse, bu küçük parçalar sistem tarafından otomatik olarak birleştirilerek LLM'e doğrudan ****büyük Parent düğümü**** gönderilir.
- Bu birleştirme süreci sorgu çalışma günlüğünde (logs) *'merging nodes into a parent node'* şeklinde gözlemlenebilir. Böylece parça parça bilgi yerine bütünsel bağlam elde edilir.

GÜNLÜK HAYAT BENZETMESİ: DEDEKTİFİN KANIT DOSYASI

Bir dedektif masasında küçük delil kırıntılarını (Child nodes) incelemektedir. Eğer elindeki 4 küçük delilin 3'ü aynı şüpheliye ve aynı klasöre işaret ediyorsa, dedektif bu küçük delillerle tek tek uğraşmayı bırakır; 'Tamam, büyük resmi gördüm, bu olay doğrudan şu ana dosyanın kendisidir' der ve doğrudan kütüphaneden o ana klasörün tamamını (Parent node) masasına getirir. Auto-Merging tam olarak bu akıllı hiyerarşik birleştirme prensibiyle çalışır.

Auto-Merging Retrieval Kurulumu:

```
# LlamaIndex ile Auto-Merging (Otomatik Birleştirme) Yapısı Kurulumu
from llama_index.core.node_parser import HierarchicalNodeParser
from llama_index.core.retrievers import AutoMergingRetriever

# 1. Hiyerarşik düğüm yapısının kurulması (Üst ve alt parça boyutları belirlenir)
node_parser = HierarchicalNodeParser.from_defaults(
    chunk_sizes=[2048, 512, 128] # Üst, orta ve alt seviye düğüm hiyerarşisi
)

# 2. İndeksleme ve Otomatik Birleştirici Tanımlanması
# TruLens ile entegrasyonu sağlanan auto_merging_index üzerinden query_engine oluşturulur
query_engine = auto_merging_index.as_query_engine()

# Sorgu çalışırken günlüklere (logs) yansıyan birleşme süreci:
# 'Merging 3 nodes into parent node...'
response = query_engine.query("How do I build a portfolio of AI projects?")
```

5. RAG Teknikleri Özet Karşılaştırma Matrisi

RAG Tekniği	Geri Çağırma Birimi	LLM Bağlam Birimi	Avantajı	TruLens Groundedness
Baseline (Temel RAG)	Sabit Chunk (örn. 512 token)	Sabit Chunk (örn. 512 token)	Kolay kurulum, hızlı test imkanı	Temel Seviye (%72)
Sentence-Window	Tekil Cümle (Granular)	Geniş Cümle Penceresi (+6 cümle)	Hassas arama, geniş bağlam koruması	Yüksek (+%8 Artış, %80)
Auto-Merging	Küçük Alt Düğüm (128 token)	Büyük Üst Düğüm (512 token)	Gerektiğinde akıllıca büyük resme geçiş	En Bütünsel Bağlam